

TinyNPU: A Sensor-Driven Low-Latency AI Accelerator for Ballistic Vision

19EC702 Project Work Phase I | Saveetha Engineering College, Dept. of ECE

Anna University | Approved by AICTE | Platform: PYNQ-Z2 (Zynq-7020 SoC)

Background & Motivation

Challenges in High-Speed Ballistic Vision

- Ballistic projectiles travel >300 m/s — a 5 ms delay equals 1.5 m of untracked motion.
- Standard CNN pipelines buffer a full frame in DDR before inference can start — fundamentally too slow.

Why Conventional Accelerators Fail

- Full-frame DDR write+read adds 3–50 ms of memory bottleneck before inference.
- PCIe and AXI overhead introduce non-deterministic OS scheduling latency.

The Streaming Solution

- TinyNPU begins inference on the first pixel row — zero frame-buffer latency.
- ROI-first compute and motion-aware scheduling reduce total MAC operations by $>60\%$.

Microarchitecture

Processing Element (PE)

INT8 Multiplier $\rightarrow$ INT32 Accumulator $\rightarrow$ Pass-Through Register
`output_psum += weight_stationary  $\times$  activation_streaming`

Systolic Array (8 $\times$ 8 Weight-Stationary)

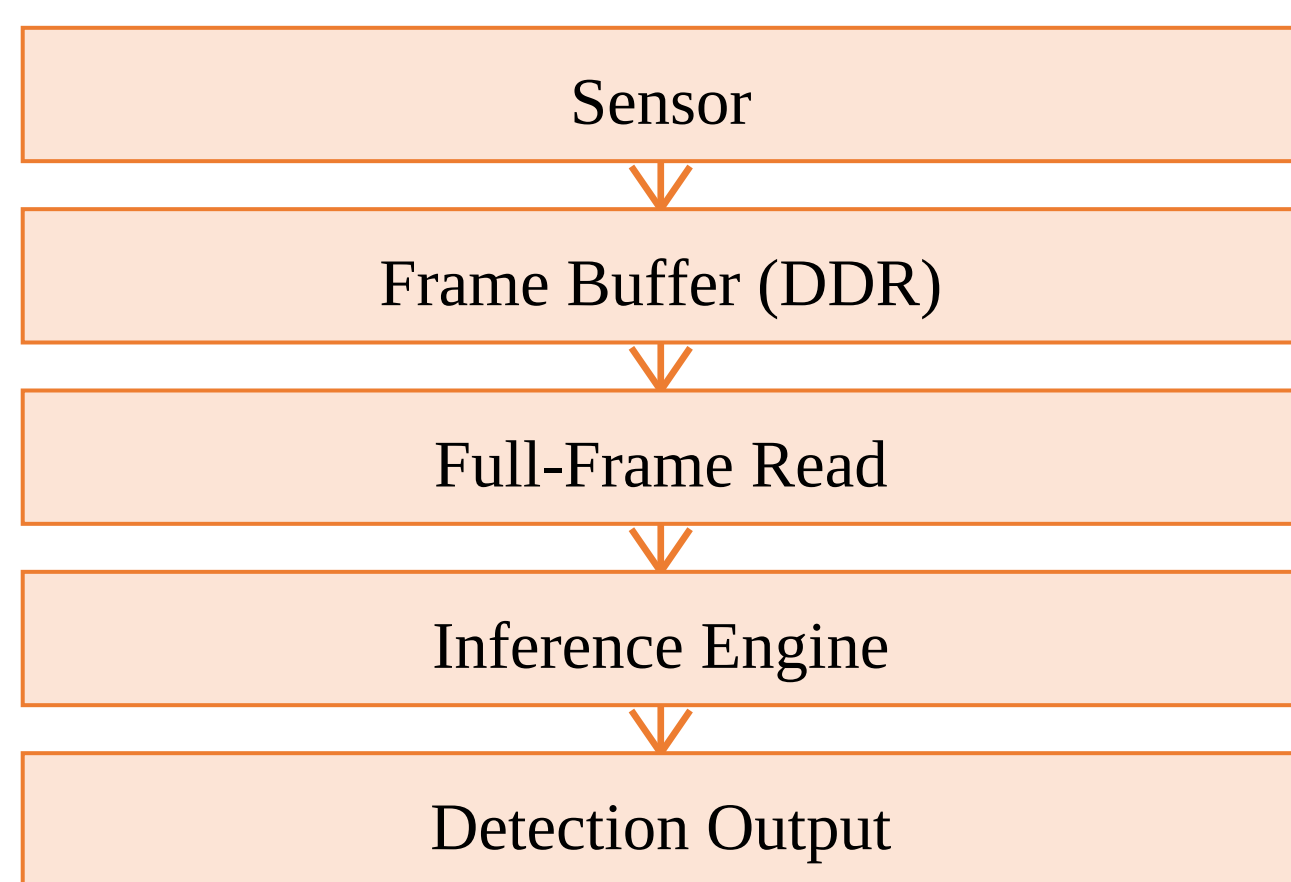
- 64 PEs compute simultaneously — one INT8 MAC per PE per clock cycle.
- Weights stay stationary in PEs; INT8 activations stream diagonally skewed.
- INT32 partial sums drain top-to-bottom and are requantized to INT8 at the output.

Memory Hierarchy

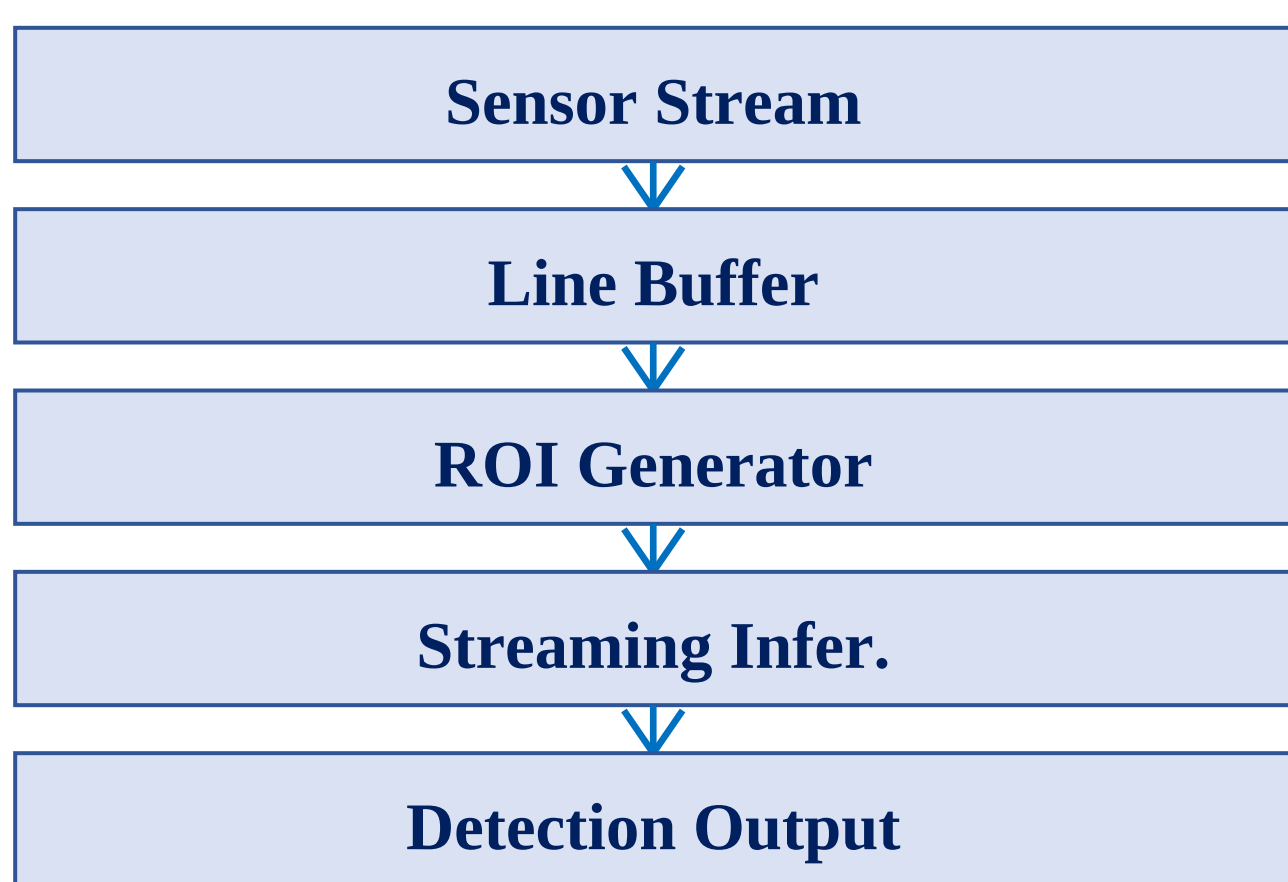
- Line Buffers enable 2D sliding-window convolution with no full-frame storage.
- Ping-Pong Buffers: Bank A loads next tile while Bank B feeds the array.

Problem Statement

Conventional Pipeline



TinyNPU Pipeline



Conventional: 5–50 ms Latency $\rightarrow$ TinyNPU Target: $< 10 \mu\text{s}$ Latency

Key Features

✓ Sensor-Driven Execution

Inference triggered by sensor stream — no frame wait.

✓ Streaming Processing

Frame buffer eliminated; compute starts on pixel row 0.

✓ Motion-Aware Scheduling

Compute dynamically focused on high-motion image regions.

✓ ROI-First Computation

Only relevant image regions processed — not the full frame.

✓ Weight-Stationary Dataflow

Weights preloaded in PEs; zero BRAM re-reads during inference.

✓ ASIC-Ready RTL

100% vendor-neutral Verilog — synthesizable to 65nm process.

TinyNPU Architecture — Full System Pipeline



Methodology

1. Literature Review

Systolic arrays, quantisation, FPGA accelerators

2. Architecture Design

Streaming-first, ROI-aware pipeline specification

3. Python Reference Model

Bit-accurate INT8 golden model for test vectors

4. RTL Development (Verilog)

Vendor-neutral, ASIC-ready synthesizable RTL

5. Verification & Simulation

Testbench + gate-level simulation (GLS)

6. FPGA Implementation

PYNQ-Z2 bitstream + Python PYNQ Overlay API

7. Performance Evaluation

Latency, throughput, resource utilisation metrics

Evaluation & Comparison

Target Performance Metrics

Sensor-to-Decision Latency	$< 10 \mu\text{s}$ (target)
Peak Throughput	25.6 GOPS @ 200 MHz
DSP48E1 Slices	64 utilized
Max Clock (FPGA)	200 MHz
Max Clock (65 nm ASIC)	500+ MHz

Qualitative Comparison

Accelerator	Streaming	ROI Support	Latency Focus
Generic FPGA CNN	No	No	Medium
FINN (Xilinx)	Yes	No	Low
NVDLA	No	No	Medium
Google TPU	No	No	High Batch
ARM Ethos-U	No	No	Medium
TinyNPU (Ours)	Yes	Yes	Ultra-Low

■ Evaluation in Progress — Numerical results pending FPGA implementation